



HOCHSCHULE
HAMM-LIPPSTADT

Project Proposal

BLE-Based Industrial Condition Monitoring System Using ESP32 Sensor Nodes

Item	Details
Course context	Advanced Embedded Systems Lab / Industrial Communication Standards Seminar
Prepared for	Prof. Dr.-Ing. Ali Hayek and Ali Alhalabi
Prepared by	Daniel Chidi Chimezie, Bertrand Mungu Cho, Stephen Uzochi Obuh, Riway Ghimire
Study program	6th Semester Electrical Engineering
Recommended topic	BLE-Based Predictive Maintenance Prototype for Temperature and Vibration Monitoring
Date	18 May 2026

Professional draft for discussion, supervisor feedback, and project approval.

Table of Contents

1. Executive Summary
2. Proposed Topic and Scope
3. Academic and Industrial Relevance
4. System Concept
5. Requirements
6. Use Case Model
7. Sequence Model
8. Implementation Plan
9. Test and Validation Plan
10. Distinction-Oriented Enhancements
11. Deliverables
12. References

1. Executive Summary

This proposal describes a laboratory project for an embedded wireless monitoring system. The project uses one ESP32-based sensor node to acquire industrially relevant measurements and a second ESP32-based gateway to receive the measurements over Bluetooth Low Energy (BLE). The gateway displays sensor values, connection status, and alarm messages through a PC serial terminal. The recommended distinction-level variant is a predictive-maintenance demonstrator using temperature and vibration monitoring.

The project is framed as an industrial communication and monitoring problem instead of a basic microcontroller exercise. It combines embedded firmware, sensor interfacing, BLE communication, threshold-based decision logic, documentation, and validation. BLE is appropriate for periodic short messages from low-power embedded devices [1], while the ESP32 platform integrates microcontroller functionality with Wi-Fi and Bluetooth/Bluetooth LE capability [2], [3].

Decision point	Recommended choice
Main project title	Design and Implementation of a BLE-Based Wireless Sensor Network for Industrial Condition Monitoring Using ESP32
Best demonstration variant	Predictive maintenance prototype using temperature and vibration monitoring
Minimum implementation	One ESP32 sensor node, one ESP32 gateway, one physical sensor, BLE data transfer, serial output
Strong implementation	Temperature + vibration sensing, alarm thresholds, reconnection handling, CSV logging or dashboard
Assessment value	Clear industrial use case, documentation quality, demonstrable communication behavior, testable requirements

2. Proposed Topic and Scope

2.1 Proposed Topic

BLE-Based Predictive Maintenance Prototype Using ESP32 Sensor Nodes for Temperature and Vibration Monitoring

2.2 Project Scope

- Develop an ESP32 sensor node that samples at least one physical variable; the recommended strong version samples both temperature and vibration.
- Implement BLE communication between a sensor node and a gateway using a defined service/characteristic model.
- Implement a gateway parser that receives structured packets, evaluates thresholds, and outputs readable status messages.
- Document the system with requirements, a use case diagram, a sequence diagram, a test plan, and a short final demonstration report.
- Keep the first prototype realistic for a four-person team, but design the architecture so that multiple nodes can be added later.

3. Academic and Industrial Relevance

The project aligns with advanced embedded systems because it requires firmware development, sensor interfacing, communication design, and systematic testing. It aligns with industrial communication standards because the BLE service/characteristic model must be defined and validated. It also has a direct Industrial IoT connection through condition monitoring and predictive-maintenance use cases.

For the vibration component, the MPU-6050 is a suitable low-cost demonstration sensor because it integrates a 3-axis accelerometer and a 3-axis gyroscope [4]. For environmental or machine-temperature measurement, the DHT22/AM2302 family can be used as a simple temperature and humidity sensor with a digital single-bus output [5].

Course theme	How the project demonstrates it
Embedded systems	ESP32 firmware, sensor readout, BLE tasks, and gateway logic
Wireless sensor networks	Sensor node and gateway architecture with possible extension to multiple nodes
Industrial communication	Defined wireless data model, periodic messages, connection state, and reconnection behavior
IoT / industrial application	Condition monitoring, early warning, and optional logging/dashboard
Technical documentation	Requirements, UML-style diagrams, test cases, and presentation-ready architecture

4. System Concept

4.1 Architecture Overview

The architecture contains a sensor layer, an ESP32 sensor-node layer, a BLE communication layer, an ESP32 gateway layer, and a PC output layer. In the minimum version, the PC output is the serial terminal. In the extended version, the PC also logs values to CSV or displays them in a Python dashboard.

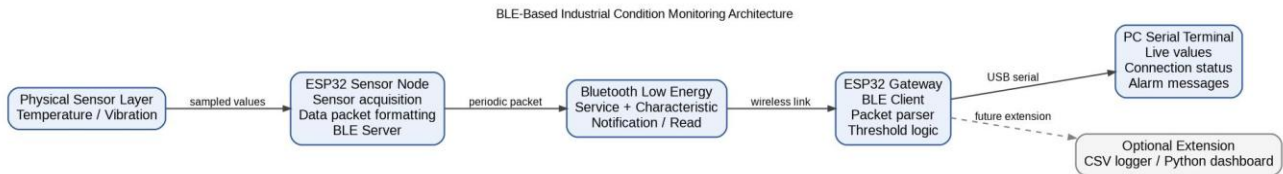


Fig. 1. System architecture for the BLE-based industrial condition monitoring prototype. Source: self-generated by the project team.

Demonstration Setup



Demo: change temperature/vibration, exceed a threshold, and show live warning output.

Fig. 2. Suggested laboratory demonstration setup. Source: self-generated by the project team.

4.2 Proposed BLE Data Format

A human-readable packet format is recommended for the first prototype because it is easy to debug during live demonstration:

```

NODE_ID:01;TEMP:28.4;VIB:0.73;STATUS:NORMAL
NODE_ID:01;TEMP:46.2;VIB:1.82;STATUS:CRITICAL
  
```

Field	Example	Meaning
NODE_ID	01	Unique identifier of the sensor node
TEMP	28.4	Measured temperature in degrees Celsius
VIB	0.73	Vibration metric from accelerometer or vibration module
STATUS	NORMAL / WARNING / CRITICAL	Gateway evaluation after threshold checking

5. Requirements

5.1 Functional Requirements

ID	Requirement
FR-01	The sensor node shall acquire at least one physical measurement from a connected sensor.
FR-02	The sensor node shall transmit the measurement to the gateway using BLE.
FR-03	The gateway shall receive, parse, and validate the sensor packet.

FR-04	The gateway shall display the latest value, node ID, connection state, and timestamp/status through the serial terminal.
FR-05	The gateway shall compare received values with configured warning and critical thresholds.
FR-06	The gateway shall generate a visible warning or critical message when a threshold is exceeded.
FR-07	The gateway shall detect BLE disconnection and attempt reconnection.
FR-08	The system shall support a configurable sampling period, initially 1 s.
FR-09	The final demonstration shall include at least one repeatable test where the alarm condition is triggered.

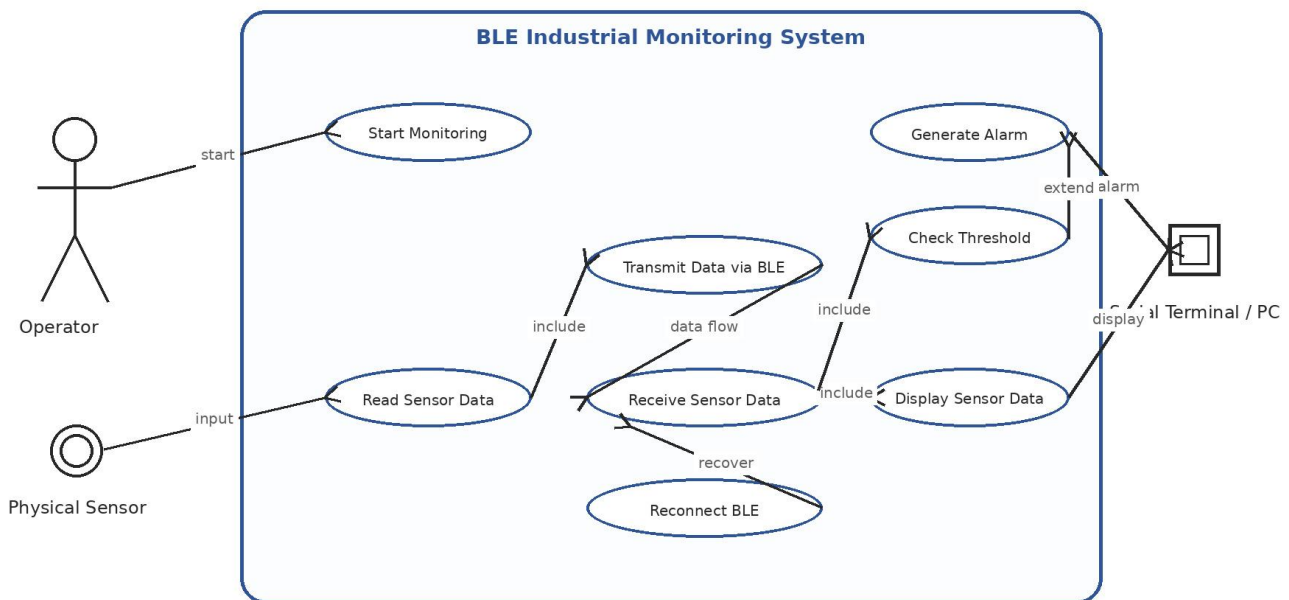
5.2 Non-Functional Requirements

ID	Requirement
NFR-01	The solution should be modular: sensor acquisition, BLE communication, parsing, threshold logic, and output should be separated in software.
NFR-02	The solution should be explainable and testable with clear serial output and documented test cases.
NFR-03	The BLE link should be stable enough for a live classroom demonstration over short range.
NFR-04	The design should be scalable to at least two sensor nodes as a future extension.
NFR-05	The report should use consistent engineering notation, numbered figures, and IEEE-style references.
NFR-06	The project should show industrial relevance through condition monitoring, machine-state observation, or environmental supervision.

6. Use Case Model

The use case model identifies the operator, the physical sensor, the PC terminal, and the core system behaviors. UML is commonly used to visualize, specify, construct, and document software and system artifacts [6].

Use Case Diagram - BLE Industrial Monitoring System



Source: self-generated by the project team; use-case content follows the earlier generated proposal diagram.

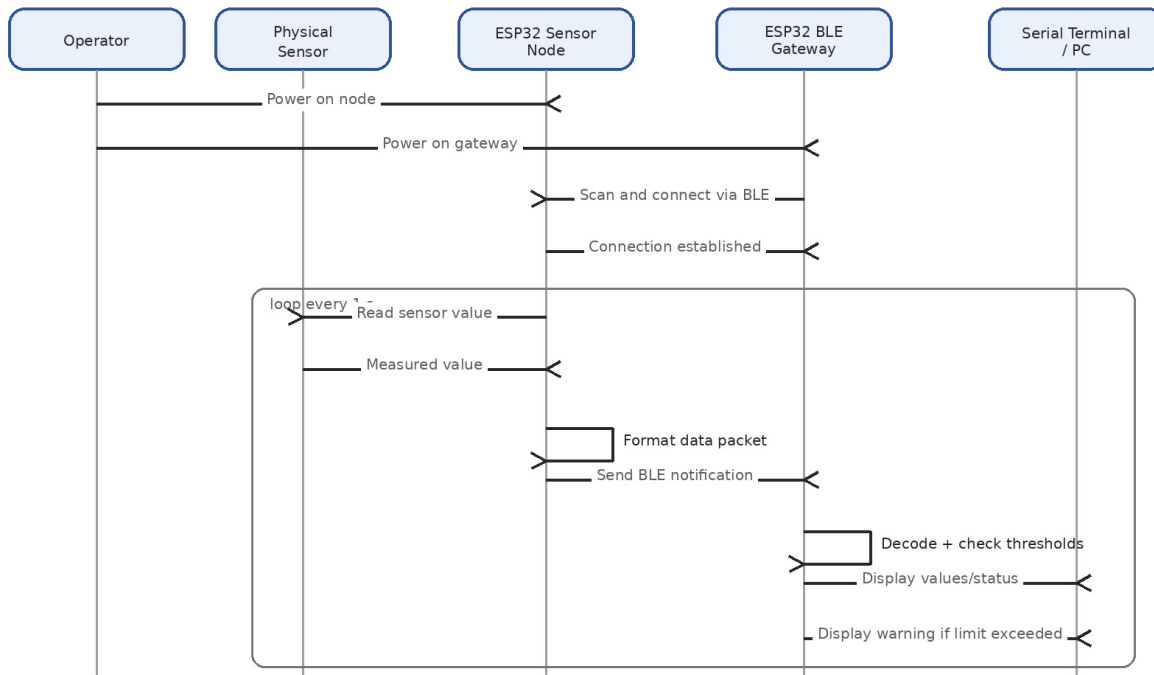
Fig. 3. Use case diagram for the BLE industrial monitoring system. Source: self-generated by the project team; content follows the earlier generated use-case model.

Use case	Primary actor	Success condition
Start monitoring	Operator	Node and gateway are powered and ready.
Read sensor data	Sensor node	A valid sensor value is available in firmware.
Transmit data via BLE	ESP32 sensor node	Gateway receives the defined packet.
Display sensor data	ESP32 gateway / PC	Terminal shows node ID, sensor values, and status.
Check threshold	ESP32 gateway	Normal, warning, or critical state is assigned.
Generate alarm	ESP32 gateway	Warning or critical message is displayed; optional LED/buzzer is activated.
Reconnect BLE	ESP32 gateway	Gateway detects disconnection and restores communication.

7. Sequence Model

The sequence model shows normal operation: start-up, connection, sensor sampling, data formatting, BLE transmission, gateway parsing, threshold checking, and output generation.

Sequence Diagram - Normal BLE Monitoring Operation



Source: self-generated by the project team based on the proposed BLE monitoring workflow.

Fig. 4. Sequence diagram for normal monitoring operation. Source: self-generated by the project team.

8. Implementation Plan

8.1 Team Distribution

Team member role	Main responsibility	Concrete output
Student 1	Sensor-node hardware and sensor interface	Wiring, sensor-reading function, calibration notes, sample data
Student 2	BLE server/client communication	BLE service/characteristic design, connection routine, notification/read function
Student 3	Gateway parser, alarm logic, and serial output	Packet parser, threshold function, readable terminal messages
Student 4	Documentation, diagrams, test cases, and presentation	Requirements, use case diagram, sequence diagram, test plan, final report

8.2 Implementation Milestones

Milestone	Description	Acceptance criterion
M1	Bring up ESP32 boards and read one sensor	Stable sensor values printed locally.
M2	Implement BLE connection	Gateway connects to node and receives a test value.
M3	Integrate real sensor packet	Gateway receives node ID and live sensor data.
M4	Add threshold evaluation	Normal, warning, and critical messages are correct.
M5	Add reconnection behavior	Gateway recovers after node restart.
M6	Prepare final documentation and demonstration	Report, diagrams, test table, and live demo are complete.

9. Test and Validation Plan

Test ID	Test case	Expected result
T-01	Power-on test for both ESP32 boards	Both boards start and print initialization messages.
T-02	Sensor acquisition test	Node prints plausible sensor values at the configured period.
T-03	BLE discovery and connection test	Gateway finds the sensor node and reports connected status.
T-04	BLE data transfer test	Gateway receives the full packet without corrupted fields.
T-05	Threshold normal test	Gateway reports NORMAL when values are below limits.
T-06	Threshold alarm test	Gateway reports WARNING or CRITICAL when a limit is exceeded.
T-07	Disconnection test	Gateway reports disconnected when the node is powered off.
T-08	Reconnection test	Gateway reconnects after the node is powered on again.
T-09	Presentation test	The team can explain architecture, diagram flow, and test result within the allocated time.

10. Distinction-Oriented Enhancements

To move the project beyond the minimum requirement, the team should add at least two enhancement features:

- Dual-sensor monitoring: combine temperature and vibration in the same BLE packet.
- Connection robustness: demonstrate loss detection and automatic reconnection.
- CSV logging: store timestamped values on the PC for later analysis.
- Simple dashboard: show current value, status, and alarm state in a Python interface.
- Multi-node extension: add a second ESP32 sensor node if time allows.
- Structured evaluation: include a test-result table with measured latency, packet period, and alarm response behavior.

11. Deliverables

1. Working ESP32 sensor-node firmware.
2. Working ESP32 gateway firmware.
3. Defined BLE service/characteristic or data packet specification.
4. Serial terminal demonstration with live sensor values and alarm state.
5. Use case diagram and sequence diagram as documentation figures.
6. Short final report including requirements, implementation summary, test results, and references.
7. Presentation slides with system architecture, live demo procedure, and lessons learned.

12. References

- [1] Bluetooth SIG, "The Bluetooth Low Energy Primer," Bluetooth SIG, version 1.2.0, Mar. 2024. [Online]. Available: <https://www.bluetooth.com/wp-content/uploads/2022/05/the-bluetooth-le-primer-v1.2.0.pdf>
- [2] Espressif Systems, "ESP32 Series Datasheet," Espressif Systems, 2025. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
- [3] Espressif Systems, "ESP32-WROOM-32 Datasheet," Espressif Systems, version 3.6, 2025. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32-wroom-32_datasheet_en.pdf
- [4] InvenSense, "MPU-6000 and MPU-6050 Product Specification," Rev. 3.4, Aug. 2013. [Online]. Available: <https://cdn.sparkfun.com/datasheets/Components/General%20IC/PS-MPU-6000A.pdf>
- [5] Aosong Electronics, "Digital-output relative humidity & temperature sensor/module AM2302," Datasheet. [Online]. Available: <https://cdn-shop.adafruit.com/datasheets/Digital%2Bhumidity%2Band%2Btemperature%2Bsensor%2BAM2302.pdf>
- [6] Object Management Group, "Unified Modeling Language, Version 2.5.1," Dec. 2017. [Online]. Available: <https://www.omg.org/spec/UML/2.5.1/About-UML>